

**LAPORAN TUGAS AKHIR DATA MINING
METODE DECISION TREE ID3**



DOSEN PENGAMPU

Dr. Eng. I Putu Agung Bayupati, S.T.,M.T.

DISUSUN OLEH

Nyoman Wiprayanka	(2305551009)
Agus Arya Wiraguna	(2305551013)
I Ketut Arya Arinatha Wardana	(2305551015)

MATA KULIAH

Data Mining (B)

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS UDAYANA

BALI

2026

DAFTAR ISI

DAFTAR ISI	i
DAFTAR TABEL.....	iii
DAFTAR GAMBAR	iv
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan Penelitian.....	2
1.4 Manfaat Penelitian	3
1.5 Batasan Masalah.....	3
BAB II LANDASAN PUSTAKA.....	5
2.1 Teknik Data Mining	5
2.1.1 Estimasi.....	5
2.1.2 Prediksi.....	6
2.1.3 Klasifikasi	6
2.1.4 Clustering	7
2.1.5 Asosiasi	7
2.2 Decision Tree	8
2.2.1 Struktur.....	8
2.2.2 Jenis-Jenis Decision Tree	9
2.3 Algoritma Decision Tree ID3	9
2.3.1 Entropi.....	10
2.3.2 Information Gain	10
2.3.3 Perbedaan ID3 dan C4.5	11
2.4 Machine Learning.....	11
2.5 Supervised Learning.....	12
BAB III HASIL DAN PEMBAHASAN.....	13
3.1 Decision Tree ID3	13
3.1.1 Metode.....	13
3.1.2 Kode Program	15
3.1.3 Hasil	24
BAB IV PENUTUP	25

4.1	Kesimpulan	25
4.1.1	Kelebihan Decision Tree ID3	25
4.1.2	Kekurangan Decision Tree ID3	26
4.2	Saran.....	27
DAFTAR PUSTAKA		29
LAMPIRAN.....		30
LAMPIRAN 1. Tautan Google Colab Dataset Kecil		30
LAMPIRAN 2. Tautan Google Colab Dataset Besar (On Progress)		30

DAFTAR KODE PROGRAM

Kode Program 3.1 Implementasi Algoritma ID3 Manual.....	22
---	----

DAFTAR GAMBAR

Gambar 3.1 Visualisasi <i>Decision Tree</i> ID3 Manual	24
--	----

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi menyebabkan jumlah data yang dihasilkan terus meningkat dalam berbagai bidang, seperti pendidikan, kesehatan, bisnis, dan industri. Data yang tersimpan dalam jumlah besar tersebut dapat menjadi sumber informasi penting apabila diolah dan dianalisis dengan tepat. Namun, banyaknya data yang tersedia sering kali sulit dianalisis secara manual sehingga diperlukan metode untuk menggali informasi dan pola dari data tersebut.

Salah satu bidang yang digunakan untuk menemukan pola dan pengetahuan dari kumpulan data adalah data mining. Data mining merupakan proses menemukan pola, hubungan, dan informasi penting dari kumpulan data berukuran besar menggunakan metode statistik, machine learning, dan basis data (Daniel T. Larose, 2014). Salah satu teknik yang banyak digunakan dalam data mining adalah klasifikasi.

Klasifikasi merupakan teknik data mining yang digunakan untuk mengelompokkan data ke dalam kelas atau kategori tertentu berdasarkan atribut yang dimiliki. Teknik ini banyak digunakan dalam berbagai kasus, seperti prediksi, analisis keputusan, dan identifikasi pola data. Untuk melakukan klasifikasi, terdapat berbagai algoritma yang dapat digunakan, salah satunya adalah *decision tree*.

Decision tree merupakan metode klasifikasi yang membentuk struktur pohon keputusan berdasarkan atribut data. Metode ini cukup populer karena memiliki bentuk yang sederhana, mudah dipahami, dan mampu menghasilkan aturan keputusan yang jelas. Salah satu algoritma *decision tree* yang sering digunakan dalam data mining adalah Iterative Dichotomiser 3 (ID3).

Algoritma ID3 dikembangkan oleh Ross Quinlan dan bekerja dengan membangun pohon keputusan berdasarkan perhitungan entropy dan information gain. Dalam prosesnya, algoritma ID3 memilih atribut terbaik yang memiliki nilai information gain tertinggi untuk dijadikan node pada pohon keputusan. Dengan pendekatan tersebut, ID3 mampu melakukan proses klasifikasi data secara sistematis dan terstruktur.

Penerapan algoritma ID3 telah digunakan dalam berbagai penelitian klasifikasi data. Penelitian yang dilakukan oleh Irmayanti dan Mardi (2021) menunjukkan bahwa algoritma ID3 dapat digunakan untuk memprediksi mahasiswa yang berpotensi mengalami drop out berdasarkan atribut tertentu. Selain itu, penelitian oleh Ferdina (2023) menunjukkan bahwa algoritma ID3 juga dapat diterapkan dalam klasifikasi faktor risiko penyakit diabetes melitus.

Berdasarkan penelitian tersebut, algoritma ID3 memiliki kemampuan dalam melakukan proses klasifikasi data secara sistematis dan mudah dipahami. Oleh karena itu, penelitian ini dilakukan untuk mempelajari konsep, proses pembentukan decision tree, serta penerapan algoritma ID3 dalam teknik klasifikasi pada data mining.

1.2 Rumusan Masalah

Rumusan masalah merupakan hal yang menjadi fokus utama dalam suatu penelitian atau studi. Berdasarkan latar belakang penelitian maka dapat dirumuskan masalah penelitian adalah sebagai berikut.

1. Apa yang dimaksud dengan data mining dan teknik klasifikasi
2. Bagaimana konsep *decision tree* dalam proses klasifikasi data
3. Bagaimana cara kerja algoritma ID3 dalam membangun pohon keputusan
4. Bagaimana penerapan entropy dan information gain pada algoritma ID3

1.3 Tujuan Penelitian

Berdasarkan rumusan masalah penelitian, terdapat beberapa tujuan yang ingin dicapai. Berikut merupakan beberapa tujuan berdasarkan rumusan masalah penelitian.

1. Memahami konsep data mining dan teknik klasifikasi.
2. Mengetahui konsep dan struktur *decision tree*.
3. Mempelajari cara kerja algoritma ID3 dalam proses klasifikasi data.
4. Memahami penerapan entropy dan information gain pada algoritma ID3.

1.4 Manfaat Penelitian

Manfaat penelitian yang diharapkan dari perancangan laporan menggunakan metode *Decision tree* ID3 adalah sebagai berikut.

1. Manfaat Akademis

Penelitian ini diharapkan dapat menambah wawasan dan pemahaman mengenai penerapan data mining, khususnya teknik klasifikasi menggunakan algoritma ID3.

2. Manfaat Praktis

Penelitian ini diharapkan dapat menjadi referensi dalam penerapan algoritma ID3 untuk proses klasifikasi data pada berbagai permasalahan data mining.

3. Manfaat Bagi Penulis

Penelitian ini dapat meningkatkan pemahaman penulis mengenai konsep data mining, *decision tree*, dan algoritma ID3 dalam proses pengolahan dan analisis data.

1.5 Batasan Masalah

Agar pembahasan dalam penelitian ini lebih terarah dan tidak menyimpang dari tujuan penelitian, maka ditetapkan beberapa batasan masalah sebagai berikut:

1. Penelitian berfokus pada penerapan data mining menggunakan teknik klasifikasi.
2. Algoritma utama yang dibahas dalam penelitian ini adalah algoritma ID3 (Iterative Dichotomiser 3).
3. Penelitian membahas perbandingan konsep dasar antara algoritma ID3 dan C4.5 secara umum, namun implementasi dan pembentukan *decision tree* difokuskan pada algoritma ID3.
4. Proses klasifikasi dilakukan menggunakan metode *decision tree* berdasarkan perhitungan entropy dan information gain.
5. Dataset yang digunakan telah disesuaikan untuk proses klasifikasi menggunakan algoritma ID3.
6. Penelitian difokuskan pada proses pembentukan pohon keputusan serta analisis hasil klasifikasi data.

7. Penelitian tidak membahas optimasi lanjutan seperti pruning kompleks, ensemble method, maupun pengembangan *decision tree* lainnya secara mendalam.

BAB II

LANDASAN PUSTAKA

2.1 Teknik Data Mining

Data mining merupakan proses untuk menemukan pola, hubungan, dan kecenderungan yang bermanfaat dari kumpulan data berukuran besar. Menurut Larose dan Larose, data mining digunakan untuk menggali informasi yang sebelumnya tersembunyi di dalam data, sehingga data tersebut dapat diubah menjadi pengetahuan yang berguna dalam pengambilan keputusan. Data mining tidak hanya sekadar mengolah data, tetapi juga berupaya menemukan pola yang dapat dijelaskan, dipahami, dan dimanfaatkan sesuai kebutuhan analisis.

Dalam penerapannya, data mining memiliki beberapa tugas atau teknik utama, antara lain estimasi, prediksi, klasifikasi, clustering, dan asosiasi. Masing-masing teknik memiliki tujuan yang berbeda, misalnya memperkirakan nilai numerik, meramalkan kejadian masa depan, mengelompokkan data ke dalam kelas tertentu, membentuk kelompok berdasarkan kemiripan, serta menemukan hubungan antaratribut dalam data. Teknik-teknik tersebut membantu analisis dalam memahami karakteristik data dan menghasilkan informasi yang lebih bernilai (Daniel T. Larose, 2014).

2.1.1 Estimasi

Estimasi merupakan teknik data mining yang digunakan untuk memperkirakan nilai dari suatu variabel target yang bersifat numerik. Dalam teknik ini, nilai target diperkirakan berdasarkan sejumlah variabel prediktor, baik yang berbentuk numerik maupun kategorikal. Model estimasi dibangun menggunakan data yang sudah lengkap, yaitu data yang memiliki nilai target dan nilai prediktor, kemudian model tersebut digunakan untuk memperkirakan nilai target pada data baru.

Contoh penerapan estimasi dapat ditemukan pada perkiraan pendapatan seseorang berdasarkan usia, tingkat pendidikan, pekerjaan, atau pengalaman kerja. Estimasi juga dapat digunakan untuk memperkirakan nilai penjualan, jumlah biaya, atau tingkat risiko tertentu. Dengan demikian, estimasi membantu pengguna data dalam memperoleh gambaran nilai yang belum diketahui berdasarkan pola yang terdapat pada data sebelumnya.

2.1.2 Prediksi

Prediksi merupakan teknik data mining yang digunakan untuk memperkirakan kejadian atau nilai yang kemungkinan terjadi pada masa mendatang. Teknik ini memiliki kemiripan dengan estimasi, tetapi prediksi lebih menekankan pada aspek waktu, yaitu bagaimana pola data saat ini atau masa lalu dapat digunakan untuk memperkirakan kondisi di masa depan.

Contohnya, prediksi dapat digunakan untuk memperkirakan apakah pelanggan akan berhenti menggunakan layanan, apakah penjualan bulan depan akan meningkat, atau apakah seorang nasabah berpotensi mengalami kredit macet. Dengan adanya prediksi, organisasi dapat menyusun langkah antisipatif sebelum suatu kejadian benar-benar terjadi.

2.1.3 Klasifikasi

Klasifikasi merupakan teknik data mining yang bertujuan untuk menempatkan suatu data ke dalam kategori atau kelas tertentu yang telah ditentukan sebelumnya. Dalam klasifikasi, model dibangun menggunakan data yang sudah memiliki label kelas, kemudian model tersebut digunakan untuk mengklasifikasikan data baru berdasarkan pola atribut yang dimilikinya. Larose dan Larose menempatkan klasifikasi sebagai salah satu tugas utama dalam data mining, serta membahas beberapa metode klasifikasi seperti *k-nearest neighbor*, *decision tree*, *neural network*, *logistic regression*, *naive Bayes*, dan *Bayesian networks*.

Tahapan klasifikasi dapat disesuaikan dengan proses data mining, yaitu dimulai dari pengumpulan dan pemahaman data, persiapan data, pembentukan model, pengujian model, hingga evaluasi model. Pada tahap persiapan data, data perlu dibersihkan, dipilih variabelnya, dan disesuaikan agar dapat diproses oleh metode klasifikasi. Setelah itu, model klasifikasi dibangun pada tahap modeling, kemudian hasilnya dievaluasi untuk melihat kualitas dan efektivitas model. Evaluasi model klasifikasi dapat dilakukan menggunakan ukuran seperti *accuracy*, *error rate*, *sensitivity*, *specificity*, *false-positive rate*, *false-negative rate*, serta proporsi *true positive* dan *true negative*.

Beberapa metode klasifikasi yang dibahas oleh Larose dan Larose antara lain *k-nearest neighbor* dan *decision tree*. *K-nearest neighbor* merupakan metode klasifikasi yang menentukan kelas data baru berdasarkan kedekatan jarak dengan data lain yang sudah memiliki kelas. Sementara itu, *decision tree* merupakan metode klasifikasi berbentuk struktur pohon keputusan, yang terdiri dari decision node, branch, dan leaf node. Selain itu, Larose dan Larose juga membahas metode klasifikasi lain seperti neural network, logistic regression, naïve Bayes, dan Bayesian networks. Adapun Support Vector Machine (SVM) tidak menjadi metode utama yang dibahas dalam bagian klasifikasi pada buku tersebut, sehingga lebih aman tidak dicantumkan jika laporan ingin benar-benar mengikuti isi buku Larose dan Larose.

2.1.4 Clustering

Clustering merupakan teknik data mining yang digunakan untuk mengelompokkan data berdasarkan kemiripan karakteristik antarobjek. Berbeda dengan klasifikasi, clustering tidak membutuhkan label kelas yang telah ditentukan sebelumnya. Data akan dikelompokkan secara otomatis ke dalam beberapa cluster, sehingga objek dalam satu cluster memiliki kemiripan yang tinggi, sedangkan objek pada cluster yang berbeda memiliki karakteristik yang lebih berbeda.

Contoh penerapan clustering adalah pengelompokan pelanggan berdasarkan pola pembelian, pengelompokan mahasiswa berdasarkan perilaku akademik, atau pengelompokan wilayah berdasarkan karakteristik sosial ekonomi. Teknik ini membantu analis menemukan struktur alami dalam data, terutama ketika kategori data belum diketahui sejak awal.

2.1.5 Asosiasi

Asosiasi merupakan teknik data mining yang digunakan untuk menemukan hubungan atau keterkaitan antaratribut dalam data. Teknik ini berusaha mengetahui atribut atau item apa saja yang sering muncul secara bersamaan. Larose dan Larose menjelaskan bahwa asosiasi sering digunakan dalam affinity analysis atau market basket analysis, yaitu analisis untuk mengetahui keterkaitan antarbarang atau kejadian dalam suatu kumpulan data.

Hasil dari teknik asosiasi biasanya berbentuk aturan, misalnya “jika membeli roti, maka cenderung membeli susu.” Aturan asosiasi umumnya diukur menggunakan nilai support dan confidence untuk mengetahui seberapa sering pola tersebut muncul dan seberapa kuat hubungan antaritem tersebut. Teknik ini banyak digunakan dalam bisnis, pemasaran, rekomendasi produk, dan analisis perilaku konsumen.

2.2 Decision Tree

Decision tree merupakan metode klasifikasi yang menggunakan struktur berbentuk pohon untuk membantu proses pengambilan keputusan. Dalam struktur ini, node akar menunjukkan atribut awal yang digunakan untuk membagi data, cabang menunjukkan nilai atau kondisi dari atribut tersebut, sedangkan leaf node menunjukkan hasil klasifikasi akhir. Menurut Larose dan Larose, *decision tree* termasuk metode *supervised learning* yang membutuhkan data latih dengan variabel target yang sudah diklasifikasikan sebelumnya (Daniel T. Larose, 2014).

Metode *decision tree* banyak digunakan karena hasilnya mudah dipahami dan dapat diterjemahkan menjadi aturan keputusan. Larose dan Larose menjelaskan bahwa salah satu keunggulan *decision tree* adalah kemampuannya menghasilkan *decision rules*, yaitu aturan berbentuk “jika–maka” yang diperoleh dari jalur akar hingga daun pada pohon keputusan. Dalam penelitian prediksi mahasiswa *drop out*, *decision tree* digunakan sebagai model klasifikasi dengan algoritma ID3 untuk membentuk pohon keputusan dan menghasilkan aturan prediksi berdasarkan data akademik mahasiswa.

2.2.1 Struktur

Struktur *decision tree* terdiri dari beberapa komponen utama yang saling terhubung dalam membentuk aturan keputusan. Komponen pertama adalah *root node*, yaitu simpul utama yang berada pada bagian paling atas pohon dan menjadi titik awal proses klasifikasi. *Root node* biasanya berisi atribut yang dianggap paling berpengaruh dalam membagi data ke dalam kelas tertentu.

Komponen berikutnya adalah *internal node*, yaitu simpul yang merepresentasikan atribut atau kondisi lanjutan setelah data dibagi dari *root node*. Setiap *internal node* dihubungkan oleh *branch* atau cabang yang menunjukkan nilai

atau hasil dari suatu atribut. Pada bagian akhir pohon terdapat *leaf node*, yaitu simpul akhir yang menunjukkan hasil klasifikasi atau keputusan. Dengan struktur tersebut, *decision tree* dapat menghasilkan aturan keputusan yang mudah dipahami dalam bentuk alur dari akar hingga daun.

2.2.2 Jenis-Jenis Decision Tree

Terdapat beberapa algoritma *decision tree* yang umum digunakan dalam proses klasifikasi maupun prediksi. Setiap algoritma memiliki cara kerja yang berbeda dalam menentukan atribut terbaik dan membentuk pohon keputusan. Beberapa algoritma tersebut antara lain ID3, C4.5, dan CART.

1. **ID3 (Iterative Dichotomiser 3)** : merupakan algoritma *decision tree* yang menggunakan nilai *entropy* dan *information gain* untuk memilih atribut terbaik sebagai node.
2. **C4.5**: merupakan pengembangan dari ID3 yang dapat menangani data kontinu, *missing value*, serta menggunakan konsep *gain ratio* dalam pemilihan atribut.
3. **CART (Classification and Regression Tree)** : merupakan algoritma *decision tree* yang dapat digunakan untuk klasifikasi maupun regresi, dengan pembentukan pohon menggunakan pemisahan biner atau *binary split*.

2.3 Algoritma Decision Tree ID3

Algoritma ID3 atau *Iterative Dichotomiser 3* merupakan salah satu algoritma dalam metode *decision tree* yang digunakan untuk proses klasifikasi data. Algoritma ini dikembangkan oleh J. Ross Quinlan dan bekerja dengan cara membentuk pohon keputusan secara *top-down*, yaitu dimulai dari simpul akar hingga ke simpul daun. Dalam proses pembentukan pohon keputusan, ID3 memilih atribut terbaik sebagai akar berdasarkan nilai *information gain* tertinggi. Atribut yang memiliki nilai *gain* paling besar dianggap sebagai atribut yang paling baik dalam memisahkan data ke dalam kelas tertentu.

Secara umum, tahapan kerja algoritma ID3 dimulai dengan memilih atribut sebagai akar, membuat cabang untuk setiap nilai atribut, membagi kasus ke

dalam cabang yang sesuai, kemudian mengulangi proses tersebut pada setiap cabang sampai seluruh data pada cabang memiliki kelas yang sama. Setelah pohon keputusan terbentuk, hasilnya dapat diubah menjadi aturan atau *rule* klasifikasi. Dalam penelitian prediksi mahasiswa berpotensi *drop out*, algoritma ID3 digunakan untuk membentuk pohon keputusan berdasarkan atribut seperti IPS semester 1 sampai 4 dan total SKS. Berikut merupakan beberapa rumus yang digunakan pada perhitungan menggunakan ID3 (Dede Irmayanti, 2021).

2.3.1 Entropi

Entropy digunakan untuk mengukur tingkat ketidakpastian atau ketidakmurnian suatu kumpulan data. Semakin kecil nilai *entropy*, maka data semakin homogen. Sebaliknya, semakin besar nilai *entropy*, maka data semakin beragam. Rumus *entropy* adalah sebagai berikut:

$$Entropy(S) = - \sum_{i=1}^n p_i \log_2 p_i$$

Dengan,

S = himpunan kelas

n = jumlah partisi atau kelas

p_i = proporsi jumlah data pada kelas ke – i terhadap seluruh data

2.3.2 Information Gain

Setelah nilai *entropy* diketahui, langkah berikutnya adalah menghitung *information gain*. *Information gain* digunakan untuk mengukur seberapa baik suatu atribut dalam memisahkan data ke dalam kelas tertentu. Atribut dengan nilai *information gain* tertinggi akan dipilih sebagai simpul atau node pada pohon keputusan. Rumus *information gain* adalah sebagai berikut:

$$Gain(S, A) = Entropy(S) - \sum_{i=1}^n \frac{|S_i|}{|S|} \times Entropy(S_i)$$

Dengan,

S = himpunan kelas

$A = \text{atribut}$

$S_i = \text{jumlah kasus pada partisi ke } - i$

$|S| = \text{jumlah seluruh kasus}$

$\text{Entropy}(S_i) = \text{nilai entropy pada partisi ke } - i$

Dengan demikian, algoritma ID3 bekerja dengan membandingkan nilai *gain* dari setiap atribut. Atribut dengan nilai *gain* tertinggi dipilih sebagai akar atau node karena dianggap paling mampu membedakan data ke dalam kelas yang sesuai. Proses ini dilakukan secara berulang hingga terbentuk pohon keputusan yang dapat digunakan untuk melakukan klasifikasi data baru.

2.3.3 Perbedaan ID3 dan C4.5

ID3 dan C4.5 sama-sama merupakan algoritma *Decision tree* yang digunakan untuk klasifikasi. Keduanya membentuk pohon keputusan dengan memilih atribut terbaik sebagai node berdasarkan perhitungan tertentu. Perbedaan utamanya adalah ID3 menggunakan Information Gain untuk memilih atribut terbaik, sedangkan C4.5 menggunakan pengembangan berupa Gain Ratio. C4.5 dibuat untuk memperbaiki kelemahan ID3, terutama agar pemilihan atribut tidak terlalu bias terhadap atribut yang memiliki banyak nilai. ID3 lebih sederhana dan cocok digunakan untuk memahami konsep dasar *Decision tree*, seperti entropy dan Information Gain. Sementara itu, C4.5 lebih fleksibel karena dapat menangani data numerik, missing value, dan memiliki proses pruning untuk mengurangi pohon yang terlalu kompleks.

2.4 Machine Learning

Machine learning merupakan cabang dari kecerdasan buatan (Artificial Intelligence) yang memungkinkan sistem komputer mempelajari pola dari data dan membuat keputusan secara otomatis tanpa harus diprogram secara eksplisit. Machine learning bekerja dengan memanfaatkan data sebagai dasar pembelajaran untuk menghasilkan model yang dapat digunakan dalam proses prediksi maupun klasifikasi. Penerapan machine learning saat ini banyak digunakan dalam berbagai bidang, seperti pengenalan wajah, sistem rekomendasi, deteksi spam, analisis

kesehatan, dan prediksi bisnis. Berdasarkan metode pembelajarannya, machine learning dibagi menjadi beberapa jenis, yaitu supervised learning, unsupervised learning, dan reinforcement learning.

2.5 Supervised Learning

Supervised learning merupakan metode machine learning yang menggunakan data berlabel dalam proses pelatihannya. Data berlabel adalah data yang telah memiliki target atau kelas tertentu sehingga model dapat mempelajari hubungan antara atribut dan hasil keluaran. Tujuan utama supervised learning adalah membangun model yang mampu memprediksi hasil dari data baru berdasarkan pola yang dipelajari sebelumnya. Supervised learning umumnya digunakan pada permasalahan klasifikasi dan prediksi. Beberapa algoritma yang termasuk dalam supervised learning antara lain:

1. Decision Tree
2. Naive Bayes
3. Support Vector Machine (SVM)
4. K-Nearest Neighbor (KNN)

BAB III

HASIL DAN PEMBAHASAN

3.1 Decision Tree ID3

Pada sub bab ini akan dibahas mengenai penerapan algoritma Decision Tree ID3 dalam proses klasifikasi data. Pembahasan dimulai dari metode yang digunakan, mencakup perhitungan *entropy* dan *information gain* sebagai dasar pembentukan pohon keputusan. Selanjutnya, dijelaskan implementasi algoritma dalam bentuk kode program serta hasil yang diperoleh dari proses klasifikasi. Adapun penjelasan lebih lengkapnya adalah sebagai berikut.

3.1.1 Metode

Pada penelitian ini, metode yang digunakan untuk membangun model klasifikasi adalah algoritma *Iterative Dichotomiser 3* (ID3). ID3 merupakan salah satu algoritma *decision tree* yang dikembangkan oleh Ross Quinlan pada tahun 1986 dan menjadi salah satu algoritma klasifikasi paling fundamental dalam bidang *data mining* dan *machine learning*. Algoritma ini bekerja secara *top-down* dengan pendekatan *greedy*, yaitu memilih atribut terbaik pada setiap tahap pembentukan node berdasarkan kriteria tertentu, tanpa melakukan peninjauan ulang (*backtracking*) terhadap keputusan yang telah diambil sebelumnya. Pendekatan ini membuat ID3 menjadi algoritma yang efisien karena tidak perlu mengevaluasi seluruh kemungkinan struktur pohon, melainkan langsung memilih pilihan terbaik pada setiap iterasi.

Secara konseptual, ID3 membentuk pohon keputusan secara rekursif dengan cara memilih atribut yang paling mampu memisahkan data ke dalam kelas-kelas target yang sesuai. Pohon keputusan yang dihasilkan terdiri dari beberapa komponen utama, yaitu akar (*root node*) yang merupakan atribut pertama dan paling penting dalam proses klasifikasi, simpul internal (*internal node*) yang merepresentasikan atribut-atribut pengujian berikutnya, cabang (*branch*) yang merepresentasikan nilai-nilai dari atribut, serta daun (*leaf node*) yang merepresentasikan kelas atau keputusan akhir dari proses klasifikasi. Untuk menentukan atribut mana yang akan dijadikan node pada setiap tahap, ID3 menggunakan dua konsep utama dari teori informasi, yaitu *entropy* dan *information gain*.

3.1.1.1 Entropi

Entropy digunakan untuk mengukur tingkat ketidakpastian atau ketidakmurnian suatu kumpulan data. Semakin kecil nilai *entropy*, maka data semakin homogen. Sebaliknya, semakin besar nilai *entropy*, maka data semakin beragam. Rumus *entropy* adalah sebagai berikut:

$$Entropy(S) = - \sum_{i=1}^n p_i \log_2 p_i$$

Dengan,

S = himpunan kelas

n = jumlah partisi atau kelas

p_i = proporsi jumlah data pada kelas ke – i terhadap seluruh data

3.1.1.2 Information Gain

Setelah nilai *entropy* diketahui, langkah berikutnya adalah menghitung *information gain*. *Information gain* digunakan untuk mengukur seberapa baik suatu atribut dalam memisahkan data ke dalam kelas tertentu. Atribut dengan nilai *information gain* tertinggi akan dipilih sebagai simpul atau node pada pohon keputusan. Rumus *information gain* adalah sebagai berikut:

$$Gain(S, A) = Entropy(S) - \sum_{i=1}^n \frac{|S_i|}{|S|} \times Entropy(S_i)$$

Dengan,

S = himpunan kelas

A = atribut

S_i = jumlah kasus pada partisi ke – i

$|S|$ = jumlah seluruh kasus

$Entropy(S_i)$ = nilai *entropy* pada partisi ke – i

Dengan demikian, algoritma ID3 bekerja dengan membandingkan nilai *gain* dari setiap atribut. Atribut dengan nilai *gain* tertinggi dipilih sebagai akar atau

node karena dianggap paling mampu membedakan data ke dalam kelas yang sesuai. Proses ini dilakukan secara berulang hingga terbentuk pohon keputusan yang dapat digunakan untuk melakukan klasifikasi data baru.

3.1.2 Kode Program

Pada sub bab ini akan dipaparkan implementasi algoritma *Decision Tree* ID3 menggunakan bahasa pemrograman Python, mulai dari pemuatan *dataset*, perhitungan *entropy* dan *information gain*, pembentukan pohon keputusan, hingga proses prediksi. Berikut merupakan kode program implementasi algoritma *Decision Tree* ID3 secara manual pada *dataset* tennis.

```
# =====
# 1. IMPORT LIBRARY
# =====
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# =====
# 2. LOAD DATA TENNIS DARI PAPER
# =====
data = [
    {"Outlook": "Sunny", "Temperature": "Hot", "Humidity":
    "High", "Windy": "False", "Play": "No"},
    {"Outlook": "Sunny", "Temperature": "Hot", "Humidity":
    "High", "Windy": "True", "Play": "No"},
    {"Outlook": "Cloudy", "Temperature": "Hot", "Humidity":
    "High", "Windy": "False", "Play": "Yes"},
    {"Outlook": "Rainy", "Temperature": "Mild", "Humidity":
    "High", "Windy": "False", "Play": "Yes"},
    {"Outlook": "Rainy", "Temperature": "Cool", "Humidity":
    "Normal", "Windy": "False", "Play": "Yes"},
    {"Outlook": "Rainy", "Temperature": "Cool", "Humidity":
    "Normal", "Windy": "True", "Play": "Yes"},
    {"Outlook": "Cloudy", "Temperature": "Cool", "Humidity":
    "Normal", "Windy": "True", "Play": "Yes"},
    {"Outlook": "Sunny", "Temperature": "Mild", "Humidity":
    "High", "Windy": "False", "Play": "No"},
    {"Outlook": "Sunny", "Temperature": "Cool", "Humidity":
    "Normal", "Windy": "False", "Play": "Yes"},
    {"Outlook": "Rainy", "Temperature": "Mild", "Humidity":
    "Normal", "Windy": "False", "Play": "Yes"},
    {"Outlook": "Sunny", "Temperature": "Mild", "Humidity":
    "Normal", "Windy": "True", "Play": "Yes"},
    {"Outlook": "Cloudy", "Temperature": "Mild", "Humidity":
    "High", "Windy": "True", "Play": "Yes"},
```

```

        {"Outlook": "Cloudy", "Temperature": "Hot", "Humidity":
"Normal", "Windy": "False", "Play": "Yes"},
        {"Outlook": "Rainy", "Temperature": "Mild", "Humidity":
"High", "Windy": "True", "Play": "No"},
    ]

df = pd.DataFrame(data)

print("=== DATASET TENNIS ===")
display(df)

# =====
# 3. PEMERIKSAAN DATASET
# =====
print("\n=== INFO DATASET ===")
print(df.info())

print("\n=== KOLOM DATASET ===")
print(df.columns)

# Preprocessing - Pengecekan missing value
print("\n=== MISSING VALUE ===")
print(df.isnull().sum())

display(df.describe())

# =====
# 4. RUMUS ENTROPY DAN INFORMATION GAIN
# =====
def entropy(target, nama_data="Target", verbose=True):
    values, counts = np.unique(target, return_counts=True)
    total_data = len(target)

    entropy_value = 0
    entropy_parts = []

    if verbose:
        print("\n" + "="*60)
        print(f"MENGHITUNG ENTROPY UNTUK: {nama_data}")
        print("="*60)

        print("Isi kelas target dan jumlah datanya:")
        for value, count in zip(values, counts):
            print(f"- {value}: {count} data")

        print(f"\nTotal data = {total_data}")
        print("\nProses perhitungan entropy:")

    for value, count in zip(values, counts):
        p = count / total_data
        log_p = np.log2(p)
        entropy_per_kelas = -p * log_p

        entropy_value += entropy_per_kelas
        entropy_parts.append(entropy_per_kelas)

    if verbose:

```

```

        print("\n-----")
        print(f"Kelas target      = {value}")
        print(f"Count kelas        = {count}")
        print(f"Proporsi p            = {count} / {total_data} =
{p:.4f}")
        print(f"log2(p)                = log2({p:.4f}) = {log_p:.4f}")
        print(f"Entropy kelas    = -({p:.4f}) * ({log_p:.4f})")
        print(f"                        = {entropy_per_kelas:.4f}")

    if verbose:
        print("\nHasil akhir entropy:")
        print("Entropy Total = jumlah semua entropy kelas")

        entropy_text = " + ".join([f"{part:.4f}" for part in
entropy_parts])
        print(f"Entropy Total = {entropy_text}")
        print(f"Entropy({nama_data}) = {entropy_value:.4f}")

    return float(entropy_value)

def information_gain(data, feature, target):
    print("\n" + "#" * 70)
    print(f"INFORMATION GAIN UNTUK FITUR: {feature}")
    print("#" * 70)

    total_data = len(data)

    total_entropy = entropy(
        data[target],
        nama_data=f"Total Target '{target}'",
        verbose=False
    )

    print(f"\nEntropy      awal      target      '{target}'      =
{total_entropy:.4f}")

    weighted_entropy = 0
    weighted_parts = []

    print("\n" + "=" * 70)
    print(f"MEMECAH DATA BERDASARKAN FITUR: {feature}")
    print("=" * 70)

    print(f"\n{'Value':<15}      {'Jumlah':<10}      {'Entropy':<12}
{'Bobot':<10} {'Bobot*Entropy':<15}")
    print("-" * 70)

    for value in data[feature].unique():
        subset = data[data[feature] == value]

        subset_entropy = entropy(
            subset[target],
            nama_data=f"Subset {feature} = {value}",
            verbose=False
        )

        bobot = len(subset) / total_data
        bobot_entropy = bobot * subset_entropy

```

```

        weighted_entropy += bobot_entropy
        weighted_parts.append(bobot_entropy)

    print(
        f"{str(value):<15} "
        f"{len(subset):<10} "
        f"{subset_entropy:<12.4f} "
        f"{bobot:<10.4f} "
        f"{bobot_entropy:<15.4f}"
    )

    print("-" * 70)

    print("\nPerhitungan Weighted Entropy:")
    print("Weighted Entropy = jumlah semua Bobot*Entropy")

    weighted_text = " + ".join([f"{part:.4f}" for part in
weighted_parts])
    print(f"Weighted Entropy = {weighted_text}")
    print(f"Weighted Entropy = {weighted_entropy:.4f}")

    gain = total_entropy - weighted_entropy

    print("\nRumus:")
    print("Information Gain = Entropy Awal - Weighted Entropy")

    print("\nHasil:")
    print(f"Entropy Awal          = {total_entropy:.4f}")
    print(f"Weighted Entropy          = {weighted_entropy:.4f}")
    print(f"Information Gain           = {total_entropy:.4f} - "
{weighted_entropy:.4f}")
    print(f"                          = {gain:.4f}")

    return gain

# =====
# 5. MENGHITUNG ENTROPY TOTAL DAN INFORMATION GAIN AWAL
# =====
target_column = "Play"
features = ["Outlook", "Temperature", "Humidity", "Windy"]

_ = entropy(df[target_column], nama_data="Target")

# 5.1 Menghitung Information Gain Setiap Atribut
features = ["Outlook", "Temperature", "Humidity", "Windy"]

gain_results = []

for feature in features:
    gain = information_gain(df, feature, target_column)
    gain_results.append({
        "Atribut": feature,
        "Information Gain": gain
    })

gain_df = pd.DataFrame(gain_results)

```

```

gain_df = gain_df.sort_values(by="Information Gain",
ascending=False)

print("\n== INFORMATION GAIN SETIAP ATRIBUT ==")
display(gain_df)

print("Atribut terbaik sebagai root:",
gain_df.iloc[0]["Atribut"])

# =====
# 6. MEMBUAT MODEL DECISION TREE ID3 MANUAL
# =====
def majority_class(data, target):
    return data[target].value_counts().idxmax()

def id3(data, features, target):
    # Jika semua data pada node memiliki kelas yang sama
    if len(data[target].unique()) == 1:
        return data[target].iloc[0]

    # Jika atribut sudah habis, gunakan kelas mayoritas
    if len(features) == 0:
        return majority_class(data, target)

    # Hitung information gain setiap atribut
    gains = {}

    for feature in features:
        gains[feature] = information_gain(data, feature, target)

    # Pilih atribut dengan information gain tertinggi
    best_feature = max(gains, key=gains.get)

    # Bentuk node pohon keputusan
    tree = {
        "attribute": best_feature,
        "default": majority_class(data, target),
        "branches": {}
    }

    # Atribut yang sudah dipakai tidak digunakan lagi pada cabang
    berikutnya
    remaining_features = [feature for feature in features if
feature != best_feature]

    # Buat cabang berdasarkan setiap nilai dari atribut terbaik
    for value in data[best_feature].unique():
        subset = data[data[best_feature] == value]

        tree["branches"][value] = id3(
            subset,
            remaining_features,
            target
        )

    return tree

```



```

# =====
# 7. TRAINING MODEL ID3 MANUAL
# =====
id3_tree = id3(df, features, target_column)

print("=== MODEL ID3 MANUAL ===")
print(id3_tree)

# =====
# 8. MENAMPILKAN POHON KEPUTUSAN
# =====
def count_leaves(tree):
    if not isinstance(tree, dict):
        return 1

    total = 0

    for subtree in tree["branches"].values():
        total += count_leaves(subtree)

    return total

def tree_depth(tree):
    if not isinstance(tree, dict):
        return 1

    return 1 + max(tree_depth(subtree) for subtree in
tree["branches"].values())

def plot_manual_tree(tree):
    fig, ax = plt.subplots(figsize=(18, 8))
    ax.axis("off")

    depth = tree_depth(tree)

    def draw_node(tree, x_min, x_max, y, parent_x=None,
parent_y=None, edge_label=""):
        x = (x_min + x_max) / 2

        if isinstance(tree, dict):
            label = tree["attribute"]
        else:
            label = f"Play = {tree}"

        # Membuat garis dari parent ke child
        if parent_x is not None and parent_y is not None:
            ax.plot([parent_x, x], [parent_y, y], linewidth=1)
            ax.text(
                (parent_x + x) / 2,
                (parent_y + y) / 2,
                edge_label,
                ha="center",
                va="center",
                fontsize=10
            )

```

```

        # Membuat node
        ax.text(
            x,
            y,
            label,
            ha="center",
            va="center",
            fontsize=11,
            bbox=dict(
                boxstyle="round,pad=0.4",
                fc="white",
                ec="black"
            )
        )

    # Jika node masih berupa dictionary, lanjutkan menggambar cabangnya
    if isinstance(tree, dict):
        total_leaves = count_leaves(tree)
        current_x = x_min

        for value, subtree in tree["branches"].items():
            leaves = count_leaves(subtree)
            width = (x_max - x_min) * leaves / total_leaves

            draw_node(
                subtree,
                current_x,
                current_x + width,
                y - 1 / depth,
                parent_x=x,
                parent_y=y,
                edge_label=str(value)
            )

            current_x += width

        draw_node(tree, 0, 1, 1)

    plt.title("Visualisasi Decision Tree ID3 Manual")
    plt.show()

plot_manual_tree(id3_tree)

# =====
# 9. MENGUBAH POHON KEPUTUSAN MENJADI ATURAN IF-THEN
# =====
def extract_rules(tree, conditions=None):
    if conditions is None:
        conditions = []

    if not isinstance(tree, dict):
        rule = "IF " + " AND ".join(conditions) + " THEN Play = " + tree
        return [rule]

```

```

rules = []
attribute = tree["attribute"]

for value, subtree in tree["branches"].items():
    new_conditions = conditions + [f"{attribute} = {value}"]
    rules.extend(extract_rules(subtree, new_conditions))

return rules

rules = extract_rules(id3_tree)

print("=== ATURAN IF-THEN HASIL ID3 ===")
for i, rule in enumerate(rules, start=1):
    print(f"{i}. {rule}")

# =====
# 10. PREDIKSI MENGGUNAKAN TREE MANUAL
# =====
def predict_one(tree, sample):
    if not isinstance(tree, dict):
        return tree

    attribute = tree["attribute"]
    value = sample[attribute]

    if value in tree["branches"]:
        return predict_one(tree["branches"][value], sample)
    else:
        return tree["default"]

def predict(tree, data):
    predictions = []

    for _, row in data.iterrows():
        prediction = predict_one(tree, row)
        predictions.append(prediction)

    return predictions

```

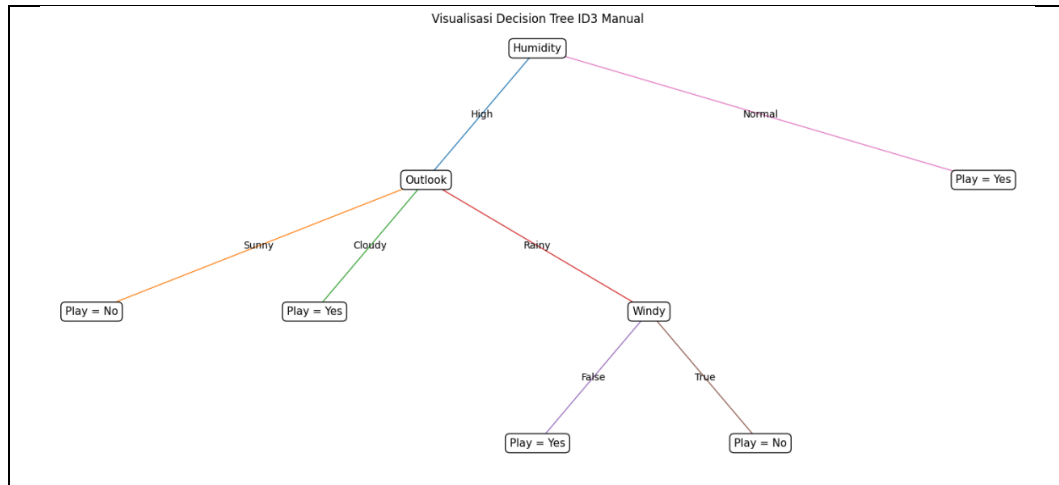
Kode Program 3.1 Implementasi Algoritma ID3 Manual

Kode program di atas merupakan implementasi lengkap algoritma *Decision Tree ID3* secara manual pada dataset tennis. Tahap awal diawali dengan pemanggilan *library* yang dibutuhkan seperti *pandas* untuk pengolahan data dalam bentuk *DataFrame*, *numpy* untuk operasi numerik seperti perhitungan logaritma pada *entropy*, *matplotlib.pyplot* untuk visualisasi pohon keputusan, serta beberapa modul dari *sklearn.metrics* yang disiapkan untuk evaluasi model. Selanjutnya, dataset tennis dimuat secara manual ke dalam bentuk *list of dictionary* lalu dikonversi menjadi *DataFrame* yang terdiri dari 14 baris data dengan empat atribut yaitu *Outlook*, *Temperature*, *Humidity*, dan *Windy*, serta satu kolom target *Play*.

Setelah itu dilakukan pemeriksaan dataset menggunakan *df.info()*, *df.columns*, *df.isnull().sum()*, dan *df.describe()* untuk memastikan struktur data sudah benar dan tidak terdapat *missing value*. Pada tahap inti, didefinisikan fungsi *entropy()* yang menghitung nilai *entropy* berdasarkan rumus penjumlahan proporsi tiap kelas dikalikan logaritma basis dua dari proporsi tersebut, dan fungsi *information_gain()* yang menghitung selisih antara *entropy* awal dengan *weighted entropy* dari setiap nilai atribut. Kedua fungsi ini dipanggil untuk menghitung *entropy* total kolom *Play* serta *information gain* dari setiap atribut, lalu hasilnya diurutkan dari nilai terbesar agar atribut dengan *information gain* tertinggi dapat dijadikan sebagai *root node* pada pohon keputusan.

Kode program di atas juga membentuk model pohon keputusan melalui fungsi *id3()* yang bekerja secara rekursif dengan tiga kondisi utama, yaitu menghentikan proses jika seluruh data memiliki kelas yang sama, menggunakan kelas mayoritas dari fungsi *majority_class()* jika atribut sudah habis, atau memilih atribut dengan *information gain* tertinggi sebagai *node* berikutnya jika kondisi tersebut belum terpenuhi. Hasil pelatihan model disimpan dalam variabel *id3_tree* berbentuk *dictionary* bersarang yang merepresentasikan setiap *node*, cabang, dan daun dari pohon keputusan. Pohon keputusan tersebut kemudian divisualisasikan menggunakan *matplotlib* melalui fungsi *plot_manual_tree()* dengan bantuan fungsi *count_leaves()* dan *tree_depth()* untuk menentukan posisi serta tata letak setiap *node*. Selain divisualisasikan, pohon keputusan juga diubah menjadi aturan klasifikasi berbentuk *IF-THEN* menggunakan fungsi *extract_rules()* yang menelusuri pohon secara rekursif dari *root* hingga daun. Pada tahap akhir, didefinisikan fungsi *predict_one()* untuk memprediksi satu sampel data dengan menelusuri pohon hingga mencapai daun dan fungsi *predict()* untuk melakukan prediksi pada seluruh data, di mana jika ditemui nilai atribut yang tidak terdapat pada cabang pohon maka digunakan nilai *default* berupa kelas mayoritas sebagai mekanisme penanganan kasus tak dikenal.

3.1.3 Hasil



Gambar 3.1 Visualisasi *Decision Tree* ID3 Manual

Gambar di atas merupakan visualisasi pohon keputusan (*decision tree*) yang dibentuk menggunakan algoritma ID3 pada dataset *Tennis*. Pohon ini memiliki atribut *Humidity* sebagai akar karena memiliki nilai *Information Gain* tertinggi dibandingkan atribut lain, yaitu sebesar 0.3705, sehingga dianggap paling mampu memisahkan data ke dalam kelas *Yes* dan *No*. Dari akar *Humidity* terdapat dua cabang, yaitu *Normal* yang langsung menghasilkan keputusan *Play = Yes*, dan *High* yang dilanjutkan ke atribut *Outlook* karena datanya masih bercampur. Pada cabang *Outlook*, kondisi *Sunny* menghasilkan keputusan *Play = No*, kondisi *Cloudy* menghasilkan keputusan *Play = Yes*, sedangkan kondisi *Rainy* masih perlu dipecah kembali menggunakan atribut *Windy*. Pada atribut *Windy*, nilai *False* menghasilkan keputusan *Play = Yes*, sedangkan nilai *True* menghasilkan keputusan *Play = No*.

Berdasarkan struktur pohon tersebut, dapat dihasilkan lima aturan *IF-THEN*, yaitu jika *Humidity = Normal* maka *Play = Yes*, jika *Humidity = High* dan *Outlook = Sunny* maka *Play = No*, jika *Humidity = High* dan *Outlook = Cloudy* maka *Play = Yes*, jika *Humidity = High* dan *Outlook = Rainy* dan *Windy = False* maka *Play = Yes*, serta jika *Humidity = High* dan *Outlook = Rainy* dan *Windy = True* maka *Play = No*. Pohon ini juga memperlihatkan bahwa atribut *Temperature* tidak muncul sama sekali dalam struktur pohon karena nilai *Information Gain*-nya paling rendah dan dianggap kurang informatif dalam membedakan kelas target. Hal ini sekaligus menunjukkan kelebihan algoritma ID3, yaitu mampu menyederhanakan model dengan hanya menggunakan atribut yang benar-benar relevan dalam proses klasifikasi.

BAB IV

PENUTUP

4.1 Kesimpulan

Berdasarkan hasil pembahasan pada bab sebelumnya mengenai penerapan algoritma *Decision Tree* ID3 dalam proses klasifikasi data, dapat ditarik beberapa kesimpulan sebagai berikut. Data mining merupakan proses untuk menemukan pola, hubungan, dan kecenderungan yang bermanfaat dari kumpulan data berukuran besar sehingga data tersebut dapat diubah menjadi pengetahuan yang berguna dalam pengambilan keputusan. Salah satu teknik utama dalam data mining adalah klasifikasi, yang bertujuan menempatkan suatu data ke dalam kategori atau kelas tertentu yang telah ditentukan sebelumnya. Untuk melakukan proses klasifikasi, dapat digunakan metode *Decision Tree* yang membentuk struktur pohon keputusan terdiri dari akar (*root node*), simpul internal (*internal node*), cabang (*branch*), serta daun (*leaf node*), dan hasilnya dapat dijelaskan dalam bentuk aturan *IF-THEN* yang mudah diinterpretasikan.

Salah satu algoritma *Decision Tree* yang menjadi fokus penelitian ini adalah Iterative Dichotomiser 3 (ID3) yang dikembangkan oleh Ross Quinlan pada tahun 1986. Algoritma ID3 bekerja secara *top-down* dengan pendekatan *greedy*, yaitu memilih atribut terbaik pada setiap tahap pembentukan *node* tanpa melakukan peninjauan ulang terhadap keputusan yang telah diambil sebelumnya. Penentuan atribut terbaik dilakukan melalui dua konsep utama dari teori informasi, yaitu *entropy* yang digunakan untuk mengukur tingkat ketidakpastian atau ketidakmurnian data, serta *information gain* yang digunakan untuk mengukur seberapa baik suatu atribut dalam memisahkan data ke dalam kelas tertentu. Atribut dengan nilai *information gain* tertinggi akan dipilih sebagai *node* pada pohon keputusan, dan proses ini dilakukan secara rekursif hingga terbentuk pohon keputusan yang dapat digunakan untuk melakukan klasifikasi data baru.

4.1.1 Kelebihan Decision Tree ID3

Berdasarkan hasil penerapan algoritma ID3 pada penelitian ini, dapat diidentifikasi beberapa kelebihan dari *Decision Tree* ID3 sebagai berikut.

1. Algoritma ID3 mampu menyederhanakan model dengan hanya menggunakan atribut yang benar-benar relevan dalam proses

klasifikasi, sehingga atribut yang memiliki nilai *Information Gain* rendah dapat diabaikan secara otomatis.

2. *Decision Tree* mudah dipahami dan mudah divisualisasikan, sehingga hasil klasifikasinya dapat ditelusuri secara intuitif dari *root node* hingga *leaf node*.
3. Hasil dari *Decision Tree* dapat dijelaskan dalam bentuk aturan *IF-THEN* yang mudah diinterpretasikan oleh pengguna, sehingga model tidak bersifat "*black box*".
4. ID3 merupakan algoritma yang sederhana dan cocok digunakan untuk memahami konsep dasar *Decision Tree*, terutama konsep *entropy* dan *Information Gain*.
5. Pendekatan *greedy* yang digunakan ID3 membuat algoritma ini efisien karena tidak perlu mengevaluasi seluruh kemungkinan struktur pohon, melainkan langsung memilih pilihan terbaik pada setiap iterasi.
6. ID3 bekerja secara sistematis dan terstruktur dalam melakukan proses klasifikasi data berdasarkan perhitungan *entropy* dan *information gain*.

4.1.2 Kekurangan Decision Tree ID3

Selain kelebihan, algoritma ID3 juga memiliki beberapa kekurangan yang teridentifikasi dari pembahasan pada penelitian ini, yaitu sebagai berikut.

1. Algoritma ID3 cenderung bias terhadap atribut yang memiliki banyak nilai. Hal ini menjadi salah satu kelemahan utama yang kemudian diperbaiki oleh algoritma pengembangnya, yaitu C4.5, melalui penggunaan *Gain Ratio*.
2. ID3 tidak dapat menangani data bertipe numerik secara langsung. Kemampuan menangani data numerik baru tersedia pada algoritma pengembangnya yaitu C4.5.
3. ID3 tidak memiliki mekanisme untuk menangani *missing value* pada data, sedangkan algoritma C4.5 mampu menanganinya.

4. ID3 tidak dilengkapi dengan proses *pruning* untuk mengurangi pohon yang terlalu kompleks, sehingga model yang dihasilkan berpotensi mengalami *overfitting* jika pohon yang terbentuk terlalu kompleks.
5. Pendekatan *greedy* tanpa *backtracking* pada ID3 membuat algoritma ini tidak melakukan peninjauan ulang terhadap keputusan yang telah diambil sebelumnya, sehingga belum tentu menghasilkan struktur pohon yang paling optimal secara global.
6. Pada penerapan ID3, ketika ditemui nilai atribut yang tidak terdapat pada cabang pohon, model harus menggunakan mekanisme penanganan berupa nilai *default* berupa kelas mayoritas untuk menangani kasus tak dikenal.

4.2 Saran

Berdasarkan hasil dan pembahasan yang telah dilakukan, terdapat beberapa saran yang dapat diberikan untuk pengembangan penelitian selanjutnya. Pertama, mengingat algoritma ID3 memiliki keterbatasan dalam menangani data numerik dan *missing value*, penelitian selanjutnya dapat mempertimbangkan penggunaan algoritma C4.5 sebagai pengembangan dari ID3 yang lebih fleksibel karena dapat menangani data numerik, *missing value*, serta memiliki proses *pruning* untuk mengurangi pohon yang terlalu kompleks. Kedua, penelitian ini hanya difokuskan pada implementasi ID3 dengan dataset yang seluruh atributnya bersifat kategorikal dan tidak memiliki *missing value*. Untuk memperoleh gambaran yang lebih komprehensif mengenai kinerja algoritma, penelitian lanjutan dapat dilakukan pada dataset yang lebih besar dan lebih kompleks. Ketiga, karena penelitian ini tidak membahas optimasi lanjutan seperti *pruning* kompleks, *ensemble method*, maupun pengembangan *decision tree* lainnya secara mendalam, hal-hal tersebut dapat menjadi topik yang menarik untuk dieksplorasi pada penelitian berikutnya. Terakhir, perbandingan kinerja antara ID3 dan algoritma klasifikasi *supervised learning* lainnya seperti *Naive Bayes*, *Support Vector Machine* (SVM), atau *K-Nearest Neighbor* (KNN) juga dapat dilakukan untuk memperkaya analisis dan

memberikan pemahaman yang lebih luas mengenai pemilihan algoritma klasifikasi yang sesuai untuk berbagai jenis permasalahan data mining.

DAFTAR PUSTAKA

- Daniel T. Larose, C. D. (2014). *Discovering Knowledge in Data: An Introduction to Data Mining*. United States: John Wiley & Sons Inc.
- Dede Irmayanti, Y. M. (2021). PREDIKSI MAHASISWA BERPOTENSI DROP OUT DENGAN METODE ITERATIF DICHOTOMISER 3 (ID3). *Jurnal Teknologi Informasi*, 103-113.
- Ferdina, N. S. (2023). PENERAPAN ALGORITMA ITERATIVE DICHOTOMISER 3 (ID3) DALAM KLASIFIKASI FAKTOR RISIKO PENYAKIT DIABETES MELITUS. *Journal of Statistics and Its Applications*, 139-146.

LAMPIRAN

LAMPIRAN 1. Tautan Google Colab Dataset Kecil

https://colab.research.google.com/drive/1T8_T66lK6T0nglen_Fc-8lffCwCAuE4k?usp=sharing

LAMPIRAN 2. Tautan Google Colab Dataset Besar (**On Progress**)

https://colab.research.google.com/drive/1T8_T66lK6T0nglen_Fc-8lffCwCAuE4k?usp=sharing